

Exp 10

Aim

To implement a Hyperledger Fabric network and demonstrate asset management using chaincode.

Theory

Hyperledger Fabric is a permissioned and modular blockchain platform designed for enterprise applications. Unlike public blockchains, participants in a Fabric network are known and authorized. The network consists of organizations, peers, ordering services and channels. A **channel** provides a private communication and transaction environment between participating organizations.

In Hyperledger Fabric, business logic is implemented using **chaincode**, which is similar to a smart contract. Chaincode defines how assets are created, accessed and modified on the blockchain ledger. In this experiment, the **basic** chaincode is used for managing digital assets.

The ledger in Fabric maintains the current state of assets in the **world state** along with the transaction history. Each asset contains attributes such as ID, color, size, owner and appraised value. Chaincode functions such as **CreateAsset**, **ReadAsset**, **UpdateAsset**, **TransferAsset**, **DeleteAsset** and **GetAllAssets** are used to perform different asset-management operations.

A transaction is proposed by a client and endorsed by the required peers. After successful endorsement, it is sent to the ordering service, which orders transactions into blocks and distributes them to the peers. The peers then validate and commit the transactions to their ledgers. This process provides controlled access, consistency and transparency while maintaining the permissioned nature of the network.

Procedure

1. Set up the Hyperledger Fabric test network.
2. Create mychannel.
3. Deploy the Go-based basic chaincode.
4. Initialize the ledger using InitLedger.
5. Read and display existing assets.
6. Create a new asset asset10.
7. Transfer its ownership.
8. Update its properties.
9. Delete the asset and verify deletion.

Output:

Screenshot 1: Fabric network and channel successfully created.

```
1.5.17: Pulling from hyperledger/fabric-ca
18afb8c8d2a4: Pull complete
6f4ebca3e823: Pull complete
b9649d3f054a: Pull complete
0b966e606d9a: Pull complete
c36bacd85367: Download complete
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-ca:1.5.17
ghcr.io/hyperledger/fabric-ca:1.5.17
==> List out hyperledger images
ghcr.io/hyperledger/fabric-baseos:2.5.16      109b4a15b282      257MB      83.2MB
ghcr.io/hyperledger/fabric-ca:1.5.17         453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.16      807493f095f6      1.01GB     257MB
ghcr.io/hyperledger/fabric-orderer:2.5.16    1d918fc7e1e0      182MB      50.7MB
ghcr.io/hyperledger/fabric-peer:2.5.16       58979b65744b      232MB      66.8MB
hyperledger/fabric-baseos:2.5                109b4a15b282      257MB      83.2MB
hyperledger/fabric-baseos:2.5.16            109b4a15b282      257MB      83.2MB
hyperledger/fabric-baseos:latest             109b4a15b282      257MB      83.2MB
hyperledger/fabric-ca:1.5                   453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                 453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                 807493f095f6      1.01GB     257MB
hyperledger/fabric-ccenv:2.5.16             807493f095f6      1.01GB     257MB
hyperledger/fabric-ccenv:latest              807493f095f6      1.01GB     257MB
hyperledger/fabric-orderer:2.5              1d918fc7e1e0      182MB      50.7MB
hyperledger/fabric-orderer:2.5.16           1d918fc7e1e0      182MB      50.7MB
hyperledger/fabric-orderer:latest            1d918fc7e1e0      182MB      50.7MB
hyperledger/fabric-peer:2.5                 58979b65744b      232MB      66.8MB
hyperledger/fabric-peer:2.5.16              58979b65744b      232MB      66.8MB
hyperledger/fabric-peer:latest               58979b65744b      232MB      66.8MB
```

Screenshot 2: basic chaincode successfully deployed.

```
Using docker and docker-compose
Generating channel genesis block 'mychannel.block'
Using organization 1
/home/chirag/hyperledger/fabric-samples/bin/configtxgen
+ '[' 0 -eq 1 ']'
+ configtxgen -profile ChannelUsingRaft -outputBlock ./channel-artifacts/mychannel.block -channelID mychannel
2026-10-07 10:12:59.658 IST 0001 INFO [common.tools.configtxgen] main -> Loading configuration
2026-10-07 10:12:59.663 IST 0002 INFO [common.tools.configtxgen.localconfig] completeInitialization -> orderer type: etcdraft
2026-10-07 10:12:59.663 IST 0003 INFO [common.tools.configtxgen.localconfig] completeInitialization -> Orderer.EtcdRaft.Options u
nset, setting to tick_interval:"500ms" election_tick:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-10-07 10:12:59.663 IST 0004 INFO [common.tools.configtxgen.localconfig] Load -> Loaded configuration: /home/chirag/hyperledg
er/fabric-samples/test-network/configtx/configtx.yaml
2026-10-07 10:12:59.664 IST 0005 INFO [common.tools.configtxgen] doOutputBlock -> Generating genesis block
2026-10-07 10:12:59.664 IST 0006 INFO [common.tools.configtxgen] doOutputBlock -> Creating application channel genesis block
2026-10-07 10:12:59.666 IST 0007 INFO [common.tools.configtxgen] doOutputBlock -> Writing genesis block
+ res=0
Creating channel mychannel
Adding orderers
+ . scripts/orderer.sh mychannel
+ '[' 0 -eq 1 ']'
+ res=0
Status: 201
{
  "name": "mychannel",
  "url": "/participation/v1/channels/mychannel",
  "consensusRelation": "consenter",
  "status": "active",
  "height": 1
}
Channel 'mychannel' created
```

Screenshot 3: docker containers running.

Screenshot 4: InitLedger transaction successful.

```

^ hyperledger/fabric-samples/test-network | main >
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com
-cert.pem" \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \
  -c '{"function":"InitLedger","Args":[]}'
2026-10-07 10:27:17.659 IST 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200

```

Screenshot 5: Existing asset retrieved using ReadAsset / GetAllAssets.

```

^ hyperledger/fabric-samples/test-network | main >
peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"function":"ReadAsset","Args":["asset1"]}'
{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5}

```

```

^ hyperledger/fabric-samples/test-network | main >
peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"function":"GetAllAssets","Args":[]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5}, {"AppraisedValue":400,"Color":"red","ID":"asset2",
"Owner":"Brad","Size":5}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10}, {"AppraisedValue":600,"
Color":"yellow","ID":"asset4","Owner":"Max","Size":10}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Siz
e":15}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15}]

```

Screenshot 6: asset10 successfully created and displayed.

```

^ hyperledger/fabric-samples/test-network | main >
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com
-cert.pem" \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \
  -c '{"function":"CreateAsset","Args":["asset10","red","10","Chirag","1000"]}'
2026-10-07 10:30:05.428 IST 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200

```

```

^ hyperledger/fabric-samples/test-network | main >
peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"function":"ReadAsset","Args":["asset10"]}'
{"AppraisedValue":1000,"Color":"red","ID":"asset10","Owner":"Chirag","Size":10}

```

Screenshot 7: Ownership of asset10 transferred to Alice.

```
^ hyperledger/fabric-samples/test-network 1/ main >
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \
  -c '{"function":"TransferAsset","Args":["asset10","Alice"]}'
2026-10-07 10:34:02.384 IST 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Chirag"
^ hyperledger/fabric-samples/test-network 1/ main >
peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"function":"ReadAsset","Args":["asset10"]}'
{"AppraisedValue":1000,"Color":"red","ID":"asset10","Owner":"Alice","Size":10}
```

Screenshot 8: asset10 successfully updated.

```
^ hyperledger/fabric-samples/test-network 1/ main >
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \
  -c '{"function":"UpdateAsset","Args":["asset10","blue","15","Alice","1500"]}'
2026-10-07 10:34:39.413 IST 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
^ hyperledger/fabric-samples/test-network 1/ main >
peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"function":"ReadAsset","Args":["asset10"]}'
{"AppraisedValue":1500,"Color":"blue","ID":"asset10","Owner":"Alice","Size":15}
```

Screenshot 9: AssetExists returned false after deletion.

```
^ hyperledger/fabric-samples/test-network 1/ main >
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile "$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \
  -c '{"function":"DeleteAsset","Args":["asset10"]}'
2026-10-07 10:35:07.021 IST 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
^ hyperledger/fabric-samples/test-network 1/ main >
peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"function":"AssetExists","Args":["asset10"]}'
false
```

Conclusion

The Hyperledger Fabric network was successfully implemented and the chaincode was used to perform asset creation, retrieval, transfer, update and deletion operations successfully.